System Integration and Architecture
Final project, building on your midterm PayMongo Checkout project

End-to-end testing with Cypress

Add an automated browser test suite to the project you already built. No new features: your job is to prove that the checkout you have actually works.

You are done when your run looks like this:

```
$ npm run test:e2e

catalog.cy.js
  ✓ E2E-01  shows the catalog with correct peso prices
  ✓ E2E-09  blocks checkout with an empty cart
orders.cy.js
  ✓ E2E-02  creates a pending order with the right total
  ✓ E2E-05  updates the order on the success page
  ✓ E2E-06  never marks a cancelled order as paid
checkout.cy.js
  ✓ E2E-03  gets a checkout URL from PayMongo
  ✓ E2E-04  hands the customer over to PayMongo
  ✓ E2E-07  shows a clear error when checkout fails
security.cy.js
  ✓ E2E-08  keeps the secret key out of the browser

9 passing   0 failing
```

Group
Your midterm group, one member per service

Time
Two weeks

Defense
Face-to-face, after intramurals

Points
100, plus up to 10 bonus

The project

Your midterm project already works: a UI, a Product Service, an Order Service, and a Payment Service that connects to PayMongo's Checkout API in test mode. For the final project you will not build new features. You will prove that the system works by adding an automated end-to-end (E2E) test suite written in Cypress.

An end-to-end test drives a real browser the way a customer would (open the shop, add an item, check out) and then checks that every service ended up in the right state. Each service can work on its own while the system as a whole still fails, and E2E tests are how you catch that.

By the end you should be able to:

- write automated browser tests with Cypress for a system made of several services;
- check results at two levels: what the page shows and what the services actually stored;
- test around a page you do not control (PayMongo's hosted checkout) with `cy.intercept()` and `cy.origin()`;
- write tests that are independent, repeatable, and free of fixed waits;
- use tests to find and fix real bugs.

What changes in your project

- Add Cypress: the `cypress/` folder, `cypress.config.js`, and an npm script.
- Add `data-cy` attributes to the elements your tests use.
- Add a way to start from known data: a seed script, or a reset endpoint enabled only in test mode.
- Fix the bugs your tests uncover.

Nothing else is required: no new services, no redesign, and no changes to the PayMongo integration.

Set up Cypress

- Install an LTS release of Node.js (Cypress 15 supports Node.js 20, 22, and 24). In your project root run `npm install --save-dev cypress`, then `npx cypress open` and choose E2E Testing.
- Use Cypress 14 or newer. The hand-over to PayMongo relies on its cross-origin behavior.
- Install it on Day 1. Cypress downloads a large browser-testing binary the first time, so do not leave this for the night before the defense.

- Seed at least three products with fixed names and prices so every run sees the same catalog. Each test creates its own orders.
- The PayMongo secret key stays in the Payment Service's `.env`, exactly as in your midterm. Tests never contain or read it. Do not use `Cypress.env()` for secrets: it copies values into the browser and has been deprecated since Cypress 15.10.

cypress.config.js

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000', // your UI
    video: true,                      // evidence for submission (off by default)
    retries: 0,                       // flaky tests must be fixed, not retried
  },
})
```

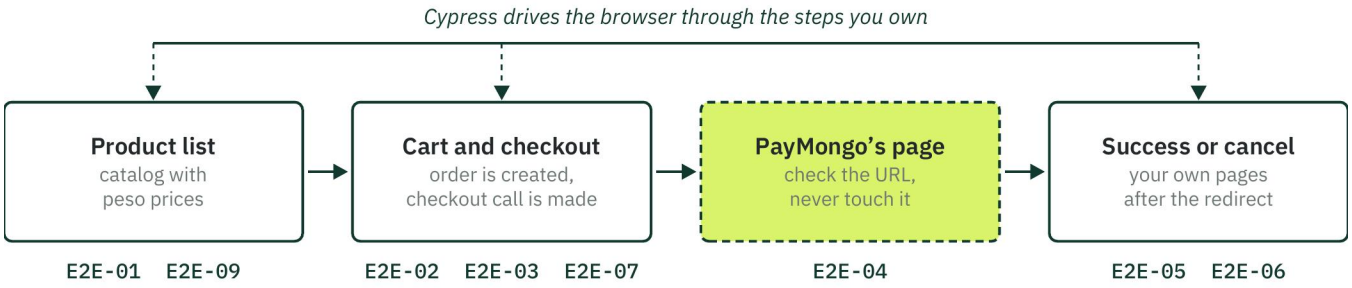
in your UI

```
<!-- give every element a test touches a stable data-cy name -->
<button data-cy="checkout-button">Checkout</button>
<span data-cy="order-status">pending</span>
```

Add `"test:e2e": "cypress run"` to `package.json` so anyone can run the whole suite headlessly with `npm run test:e2e`.

Testing the PayMongo part

The payment page belongs to PayMongo, not to you. Treat it as a boundary: test everything on your side of it, check that the customer is handed over correctly, and test your return pages directly.



The customer journey and the scenarios that cover each step. E2E-08 (no secrets in the browser) watches the whole journey.

Part of the journey	How to test it
Your pages	Drive them normally with `cy.visit()`, `cy.get()`, and `.click()`.
Your checkout call	`cy.intercept()` to watch it (status and returned `checkout_url`) or to stub it and force an error.

Part of the journey	How to test it
PayMongo's page	Only confirm you arrived: assert the hostname inside `cy.origin()`. Never type or click on it.
The redirect back	Open your success and cancel URLs with `cy.visit()`, exactly as PayMongo's redirect would.
Stored state	`cy.request()` to your Order Service API for the order's status and total.

> **Why your tests stop at PayMongo's page**
>
> Cypress's own best-practice guidance is to avoid visiting or interacting with sites you do not control. PayMongo can change its page at any time, payment forms commonly place card fields inside cross-origin frames that Cypress cannot automate, and every run would depend on the internet and on PayMongo's uptime.
>
> Also note that `cy.intercept()` only sees requests the browser makes. The call from your Payment Service to PayMongo happens server to server, so check its results through your own APIs.

Required test scenarios

Use your own routes and status names. Each scenario says what must be proven, not how your app must be built. The owner writes the test and must be able to explain it.

ID	Scenario	The test must prove	Owner
`E2E-01`	Catalog	The UI lists the seeded products with prices converted correctly from centavos (35000 → ₱350.00).	Product
`E2E-02`	Order creation	Checking out creates an order with status `pending` and the correct total, verified through the Order Service API, not just the page.	Order
`E2E-03`	Checkout session	The UI's checkout call (watched with `cy.intercept()`) succeeds and returns a `checkout_url` on `https://checkout.paymongo.com`.	Payment
`E2E-04`	Hand-over	The browser lands on `checkout.paymongo.com`. Assert the hostname inside `cy.origin()` and nothing more.	Payment
`E2E-05`	Success return	Opening the order's success URL shows the success page, and the order's status changes as your design intends (for example `paid`), in the UI and the API.	Order
`E2E-06`	Cancel return	Opening the order's cancel URL shows the cancel page; the order is never marked paid.	Order
`E2E-07`	Checkout failure	When the checkout call is stubbed to fail (status 500), the customer sees a clear error, is not redirected, and the order is not paid.	Payment
`E2E-08`	No secrets	The browser never calls `api.paymongo.com` (a spy on `cy.intercept()`), and no page or JavaScript file it receives contains `sk_test_` or `sk_live_`.	Product

ID	Scenario	The test must prove	Owner
E2E-09	Invalid checkout	Checking out with an empty cart or an invalid quantity is blocked with a clear message, and no order or checkout session is created.	Product

An example spec

Three of the scenarios in one file. Routes, ports, and `data-cy` names are examples; match them to your project.

cypress/e2e/checkout.cy.js

```
const ORDERS = 'http://localhost:3002' // Order Service API

describe('Checkout', () => {
  it('E2E-03/04: hands the customer over to PayMongo', () => {
    cy.intercept('POST', '**/checkout').as('checkout')
    cy.intercept('https://api.paymongo.com/**', cy.spy().as('browserToPayMongo'))

    cy.visit('/')
    cy.contains('[data-cy=product-card]', 'SIA1 T-Shirt')
      .find('[data-cy=add-to-cart]').click()
    cy.get('[data-cy=checkout-button]').click()

    cy.wait('@checkout').then(({ response }) => {
      expect(response.statusCode).to.eq(200)
      expect(response.body.checkout_url).to.match(/^https:\/\/checkout\.paymongo\.com\//)
    })
    cy.get('@browserToPayMongo').should('not.have.been.called')

    // PayMongo's page is another origin: confirm we arrived, touch nothing.
    cy.origin('https://checkout.paymongo.com', () => {
      cy.location('hostname').should('eq', 'checkout.paymongo.com')
    })
  })

  it('E2E-05: the success page updates the order', () => {
    cy.request('POST', `${ORDERS}/orders`, { items: [{ productId: 1, quantity: 1 }] })
      .its('body.id')
      .then((id) => {
        cy.visit(`/orders/${id}/success`) // where PayMongo sends the customer back
        cy.get('[data-cy=order-status]').should('have.text', 'paid')
        cy.request(`${ORDERS}/orders/${id}`).its('body.status').should('eq', 'paid')
      })
  })

  it('E2E-07: shows an error when checkout fails', () => {
    cy.intercept('POST', '**/checkout', { statusCode: 500, body: { error: 'Payment unavailable' } })
      .as('checkout')
    cy.visit('/')
    cy.get('[data-cy=add-to-cart]').first().click()
    cy.get('[data-cy=checkout-button]').click()
    cy.wait('@checkout')
    cy.get('[data-cy=checkout-error]').should('be.visible')
    cy.location('pathname').should('not.include', 'success')
  })
})
```

If your checkout is a regular form post that redirects, instead of a `fetch` call that returns JSON, keep the same idea: intercept that request and assert on the redirect it produces.

Rules your tests must follow

These are graded, and they are what separates a suite you can trust from one that passes by luck.

- Select elements with `data-cy` attributes, never CSS classes, element positions, or styling.
- No fixed waits such as `cy.wait(3000)`. Wait on aliased requests (`cy.wait('@checkout')`) or on assertions, which Cypress retries automatically.
- Every test is independent: it creates its own data and passes when run alone or in any order.
- Check backend state, not just the page: E2E-02, 05, 06, and 07 also read the order through the Order Service API.
- The suite passes three times in a row with `npx cypress run` and retries turned off.
- Test names start with their scenario ID, for example `it('E2E-05: ...')`, so results map to the test plan.

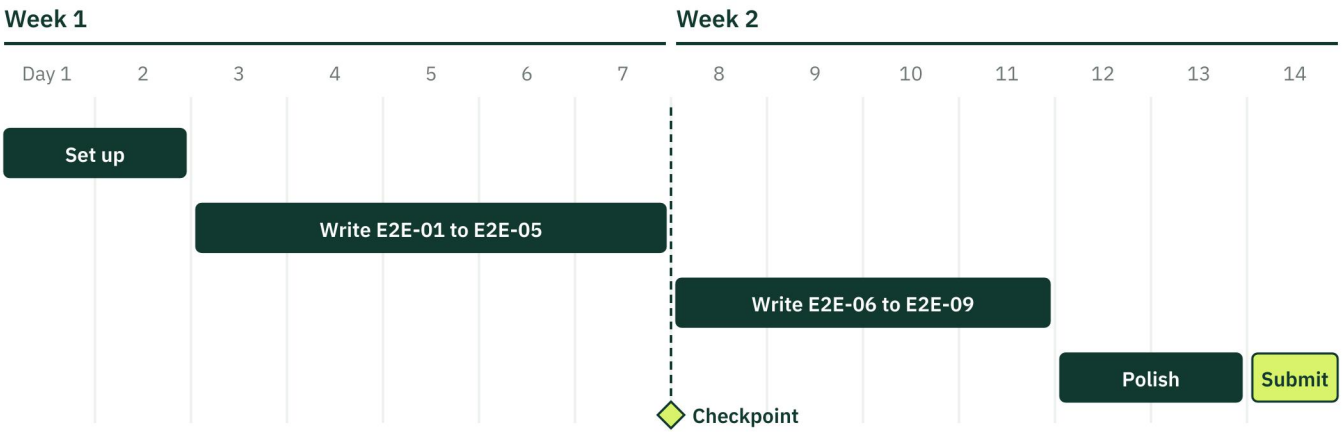
A suggested layout keeps one spec file per feature:

project layout

```
your-midterm-project/
├── ...                        # your existing UI and services, unchanged
├── cypress/
│   ├── e2e/
│   │   ├── catalog.cy.js      # E2E-01, E2E-09
│   │   ├── orders.cy.js       # E2E-02, E2E-05, E2E-06
│   │   ├── checkout.cy.js     # E2E-03, E2E-04, E2E-07
│   │   └── security.cy.js     # E2E-08
│   ├── fixtures/
│   └── support/
├── cypress.config.js
├── package.json               # "test:e2e": "cypress run"
└── README.md                  # how to start the system and run the tests
```

Two-week plan

Two weeks is enough if setup happens on Day 1. Count days from the release date and adjust to the term calendar.



- **Days 1–2, set up.** Install Cypress, add `data-cy` attributes and seed data, and write the test plan (scenario, spec file, author).
- **Days 3–7.** Write E2E-01 to E2E-05. **Checkpoint on Day 7** (submit on Canvas): the test plan and a screenshot of those five passing.
- **Days 8–11.** Write E2E-06 to E2E-09 and fix the bugs they reveal.
- **Day 14, submit** on Canvas.
- **Defense:** face-to-face after intramurals; the date will be announced. Keep your system and suite working until then.

The checkpoint is for early feedback; the grading below applies to the final submission.

What to submit

1. **Updated repository** with the Cypress suite, `data-cy` attributes, seed data, and a README section “Running the E2E tests”: start the services, seed the data, run `npm run test:e2e`.
2. **Test plan:** one table mapping E2E-01 to E2E-09 to their spec files, test names, and authors.
3. **Face-to-face defense (15–20 minutes, after intramurals):** bring a laptop with the system and the suite installed, start the system, run the suite live, then answer questions. E2E-03 and E2E-04 need internet to reach PayMongo’s sandbox.

Grading

Points are itemized like a checkout summary. Coverage points come from a live run, not from screenshots.

Test coverage	
Scenarios E2E-01 to E2E-09 are implemented and pass in a live headless run	30
Assertions prove real outcomes: what the page shows and what the services stored	10
PayMongo boundary handled correctly: `cy.intercept()` on the checkout call, `cy.origin()` only to confirm the hand-over, no automation of PayMongo's page, no secret key in test code	10
Subtotal	**50**
Test quality	
`data-cy` selectors and no fixed waits	10
Independent, repeatable tests: own data, three clean runs in a row, retries off	10
Readable suite: one spec per feature, scenario IDs in test names, README run instructions	5
Subtotal	**25**
Evidence and defense	
Test plan, Cypress run summary, and video	5
Reflection	5
Defense: live run and clear explanations of the tests	15
Subtotal	**25**
Total	**100**

Bonus, up to 10 points (the final score is capped at 100)

Add-on	Points
Server-side payment check: the success page asks the Payment Service to confirm with PayMongo (`GET /v1/checkout_sessions/{id}`, looking for a payment with status `paid`; the session's own `status` field only says `active` or `expired`), plus a test proving that opening the success URL without paying leaves the order pending. E2E-05 changes to match.	+5
CI: a GitHub Actions workflow runs the suite on every push, with the PayMongo test key stored as a repository secret for the Payment Service.	+3

Add-on	Points
Cross-browser: the suite also passes in a second browser with `cypress run --browser`.	+2

References

Cypress: Cross origin testing: https://docs.cypress.io/app/guides/cross-origin-testing

Cypress: cy.origin(): https://docs.cypress.io/api/commands/origin

Cypress: cy.intercept(): https://docs.cypress.io/api/commands/intercept

Cypress: cy.request(): https://docs.cypress.io/api/commands/request

Cypress: Best practices: https://docs.cypress.io/app/core-concepts/best-practices

Cypress: Changelog (15.10.0, Cypress.env() deprecated): https://docs.cypress.io/app/references/changelog

PayMongo: Checkout API implementation: https://docs.paymongo.com/docs/checkout-implementation

PayMongo: Get checkout session (bonus): https://docs.paymongo.com/reference/get_checkout_sessions

PayMongo: Checkout session resource (bonus): https://docs.paymongo.com/reference/checkout-session-resource